MODULE 12 · DOMAIN 2 - ML MODEL AND FM DEVELOPMENT

Choosing the modelling approach

AI service, foundation model, pre-trained model, or one you train yourself.

MODULE 12 of 43 DOMAIN 2 - Model Dev WEIGHT 24%

admin@sthithapragna.com

The most valuable question in domain two is **how little** you can build.

admin@sthithapragna.com

Five rungs, cheapest first

01

AI service

An API that already does this task.

02

Prompt an FM

Bedrock, with a well written prompt.

03

Ground or tune

Add retrieval, or fine-tune on your examples.

04

Adapt

JumpStart or a built-in algorithm on your data.

05

Train

Your own architecture, your own code.

ON THE EXAM

Module 01 introduced this. Here it becomes the actual decision procedure, with the evidence each rung requires.

AI services solve named problems outright

- **No training, no labels, no endpoints.** An API call and a per-request charge.
- Accuracy is usually good on the common case, because they were trained on enormous datasets.
- The team needs **no machine learning expertise at all**. That is often the deciding factor.
- You get their model. Limited control over edge cases and domain-specific vocabulary.
- Several support **custom entities or custom classification**, which is a middle ground.

> ON THE EXAM
>
> 'The team has no ML engineers' plus a task with a name is the AI-service signal, and it is very common.

AI services by task

TASK	SERVICE
Text, forms and tables out of documents	Amazon Textract
Sentiment, entities, language, topics in text	Amazon Comprehend
Speech to text	Amazon Transcribe
Text to speech	Amazon Polly
Translation between languages	Amazon Translate
Objects, faces, moderation in images and video	Amazon Rekognition
Personalised recommendations	Amazon Personalize
Conversational interfaces	Amazon Lex
Fraud detection on transactions	Amazon Fraud Detector

ON THE EXAM

Textract extracts, Comprehend understands. Chaining the two covers a large share of document-processing scenarios.

This tier has been pruned

- **Lookout for Vision** was discontinued in October 2025. **Lookout for Equipment** ends October 2026.
- **Amazon Kendra** moved to maintenance. Bedrock Knowledge Bases is the successor.
- Some **Comprehend and Rekognition features** moved to maintenance in the March 2026 wave.
- The services in the table above are the ones still worth reaching for.
- Foundation models absorbed several of these use cases, which is why the pruning happened.

> ON THE EXAM
>
> If an option offers a retired service name, it is either an old question or a deliberate distractor.

Foundation models on Amazon Bedrock

- Over a hundred models from around eighteen providers, behind **one API**.
- Text, code, images, video, speech and embeddings. Swap models without changing architecture.
- **Prompt engineering is a customisation technique**, and the cheapest one. Try it before anything else.
- Models run inside the AWS perimeter, governed by IAM, KMS and VPC endpoints.
- Billed per token, so cost scales with usage rather than sitting flat.

> ON THE EXAM
>
> The single API is the real architectural benefit. Changing model is a parameter change, not a rewrite.

Prompt engineering techniques worth naming

TECHNIQUE	WHAT IT DOES
Zero-shot	Instruction only. No examples. Try this first, always
Few-shot	Two to five examples in the prompt. Teaches format without training
Chain of thought	Ask for reasoning steps before the answer. Helps on multi-step problems
Role and context framing	Tell the model who it is and what it is for. Cheap and effective
Output schema	Specify the exact structure wanted, and it usually complies

ON THE EXAM

Few-shot prompting achieves much of what fine-tuning would, with no training job and no data pipeline.

admin@sthithapragna.com

Amazon Bedrock Prompt Management

- Store prompts as **versioned resources** rather than strings hard-coded in application code.
- A prompt is a production artefact. Changing one changes model behaviour, with no deployment.
- Versions let you roll back a prompt change that made quality worse.
- Prompts can carry variables, so one template serves many calls.
- Named in the C02 additions. Module 28 covers it as part of orchestration.

> ON THE EXAM
>
> An untracked prompt change is an untracked production change. Prompt versioning is the equivalent of model versioning.

Prompt caching and Intelligent Prompt Routing

- **Prompt caching** reuses a repeated prompt **prefix** between requests, skipping recomputation.
- AWS reports up to **90 percent cost saving** on cached tokens and large latency reductions.
- It suits long system prompts and fixed context that every request repeats.
- **Intelligent Prompt Routing** sends each request to a cheaper or stronger model based on the prompt.
- Neither changes model quality. Both change what you pay for the same quality.

> ON THE EXAM
>
> If a scenario has a long fixed system prompt and high request volume, caching is the intended cost answer.

What actually decides which FM

CRITERION	WHAT TO WEIGH
Task fit	Reasoning, summarisation, code, extraction, images. Families differ
Modality	Text only, or vision, or speech. This eliminates most of the list
Context window	How much you must fit in one prompt, including retrieved chunks
Latency	Smaller models answer faster. Often the real constraint
Cost per token	Input and output priced separately. Output is usually dearer
Customisation support	Not every model can be fine-tuned or distilled
Region availability	Availability differs by Region, and that is a hard constraint

ON THE EXAM

Modality and Region availability eliminate fastest. Start there, then weigh cost against quality on what remains.

Inference profiles and cross-Region inference

- A **base model** invocation goes to one Region. An **inference profile** can span several.
- Cross-Region inference routes requests across Regions in your geography, raising effective throughput.
- It helps with capacity and with bursty traffic, without you managing anything.
- **Data residency from module 08 can forbid it.** Requests leaving a Region may breach policy.
- Several newer models are only available through an inference profile, not by direct model ID.

> ON THE EXAM
>
> A genuine tension: the feature that fixes throttling is the feature a residency requirement forbids.

Batch inference on Bedrock

- Submit a JSONL file of prompts, get a file of responses back. No endpoint involved.
- **Cheaper per token** than on-demand invocation, in exchange for latency measured in time, not milliseconds.
- Right for bulk classification, bulk summarisation, and embedding a whole corpus.
- Wrong for anything a user is waiting on.
- Module 01's question again: who is waiting? If nobody is, batch is the shape.

> ON THE EXAM
>
> The same reasoning as batch transform in module 20. Real time is a requirement, not a default.

The catalogue churns faster than any course

- Models move to **Legacy** and are eventually **removed** from the catalogue.
- Already gone: Claude 2 and Instant, Llama 2, Mistral Large v1, **Titan Text G1 Express and Lite**.
- Already Legacy: Cohere Command R and R+, the Llama 3.2 family, Llama 3.1 405B.
- **Learn the selection criteria, not the model names.** Criteria survive; names do not.
- Pin the exact model ID in code, because snapshot versions update underneath you.

> ON THE EXAM
>
> A course that teaches you today's model names will be teaching you retired ones by the time you sit the exam.

SageMaker JumpStart

- A catalogue of **pre-trained models** you deploy or fine-tune on your own data.
- Covers vision, text, tabular and open foundation models, with example notebooks.
- **This is transfer learning made easy.** Someone else paid for the expensive pre-training.
- Right when the task is not a language task, so Bedrock does not apply.
- You end up with a model artefact and a SageMaker AI endpoint, so you control deployment.

> ON THE EXAM
>
> Image classification and object detection are the classic JumpStart cases. Bedrock does not cover them; JumpStart does.

Built-in algorithms

- AWS-implemented algorithms you train on your data. **No modelling code to write.**
- Optimised for SageMaker AI infrastructure, including distributed training and Pipe mode.
- The sweet spot: **tabular problems with labelled data**, where XGBoost usually wins.
- You choose the algorithm and the hyperparameters. That is real ML knowledge.
- Module 13 covers the algorithms themselves, organised by problem shape.

> ON THE EXAM
>
> Tabular prediction is where classical ML still beats foundation models decisively, and it is a large share of real work.

Training your own model

- **Script mode:** your TensorFlow or PyTorch code in an AWS-managed container.
- Needs ML expertise, labelled data, compute budget and ongoing ownership.
- Justified when the problem is genuinely novel, or the architecture is your differentiator.
- Rarely the exam's answer, because scenarios usually name a constraint that rules it out.
- If it is right, module 14 covers how the training job is actually run.

> ON THE EXAM
>
> Nothing here is wrong with training your own model. It is simply the rung that must be earned rather than assumed.

Pre-trained or custom?

PRE-TRAINED, WHEN

The problem is common

- Others have solved this exact task before.
- You have little or no labelled data.
- Time to value matters more than the last point of accuracy.
- No ML specialists on the team.

CUSTOM, WHEN

The problem is yours

- Your data has structure nobody else's has.
- You have plenty of labels already.
- A specific measurable accuracy is contractual.
- The model itself is the differentiator.

THE TIEBREAKER

Labelled data is the honest test. Without it, custom training is not a slower option; it is not an option.

What data you have decides most of this

- **No data at all:** AI service, or prompt a foundation model. Nothing else is possible.
- **Documents, no labels:** retrieval. Module 10's territory.
- **Hundreds of example pairs:** fine-tuning becomes available.
- **Thousands of labelled rows:** built-in algorithms and JumpStart open up.
- **Millions of labelled rows and specialists:** only now does custom training make sense.

> ON THE EXAM
>
> Read the data description in a scenario before reading the options. It usually eliminates half of them.

Proving the approach with a small trial

- Assemble **30 to 50 representative cases** with known good answers. That is enough to discriminate.
- Run two or three candidate approaches against them and compare on the same measure.
- Include the **hard cases deliberately**. Every approach handles the easy ones.
- A demo on three cherry-picked examples proves nothing, and is what usually happens instead.
- Bedrock Evaluations can automate the comparison. Module 18 covers it.

> ON THE EXAM
>
> The cost of a wrong approach decision is months. The cost of testing it is a day. That ratio is rarely respected.

Amazon SageMaker Canvas

- Build tabular ML models through a visual interface, with **no code and no notebooks**.
- Handles classification, regression, forecasting and time series from a spreadsheet or S3.
- It also hosts **Data Wrangler** now, from module 04.
- Models can be shared into SageMaker Studio for an engineer to take further.
- Aimed at analysts. It is the AI-service tier for problems that have no AI service.

> ON THE EXAM
>
> A useful mental slot: analysts with tabular data and no engineers. That case has no AI service, and Canvas fills it.

Combining approaches in one system

- Real systems mix rungs. Textract extracts, a tabular model scores, an FM writes the summary.
- **Route by difficulty:** a cheap model handles the common case, a strong one handles the rest.
- A confidence threshold decides the routing, exactly like the A2I pattern from module 07.
- A rules engine can handle the clear cases and only send ambiguous ones to a model.
- Each component added is another thing to monitor, version and explain.

> ON THE EXAM
>
> The exam usually asks for one approach. Production usually uses three. Recognise when a scenario is describing a pipeline.

Interpretability by approach

APPROACH	HOW WELL YOU CAN EXPLAIN A SINGLE DECISION
Linear or logistic regression	Fully. Coefficients are the explanation
Decision tree	Fully. The path through the tree is the reason
XGBoost or random forest	Feature attributions via SHAP. Good enough for most regulators
Deep neural network	Attributions only, and they are approximations
Foundation model	It can state a reason, and that reason may not be why

ON THE EXAM

A model that produces a plausible explanation is not the same as one whose decision you can audit. Regulators know the difference.

Latency shapes the choice too

- A tabular model on a real-time endpoint answers in **single-digit milliseconds**.
- A large foundation model takes **hundreds of milliseconds to seconds**, and generation is sequential.
- So a strict latency budget can rule out foundation models entirely, whatever the quality.
- Smaller models, and distillation from module 11, are the usual answers.
- AI services sit in between, and vary by task.

> ON THE EXAM
>
> Fraud scoring inside a payment authorisation has tens of milliseconds. That budget chooses the approach on its own.

Capability, and time to value

- An approach the team cannot maintain is not a solution, however good the benchmark.
- Backend developers can ship an AI service or a Bedrock call this week.
- A custom training pipeline needs someone who can debug a loss curve at two in the morning.
- **Shipping something adequate now** usually beats shipping something excellent in nine months.
- And a working simple system gives you the data to justify the complex one later.

> ON THE EXAM
>
> Scenarios name team composition for a reason. It is a constraint, not scene-setting.

Task type points at the approach

WHAT YOU ARE PREDICTING	WHERE TO LOOK
A number	Regression. Built-in algorithms, usually XGBoost
A category from a fixed set	Classification. Built-in algorithms, or Comprehend for text
A future value over time	Forecasting. DeepAR, or a time series service
Which items to show a user	Recommendation. Amazon Personalize, or Factorization Machines
Whether something is unusual	Anomaly detection. Random Cut Forest
What is in an image	Vision. Rekognition, or JumpStart, or built-in vision algorithms
Generated or summarised text	Foundation model on Bedrock

ON THE EXAM

Naming the task type is a habit worth building. It narrows the field before any AWS service is mentioned.

The problem as stated is not always the problem

- 'Predict which customers will churn' is often really 'decide who to offer a discount to'.
- The second framing has a different target, a different metric, and possibly no model at all.
- Ask what **decision** the output feeds. That is the thing being optimised.
- Ask what happens when the model is wrong in each direction. The costs are rarely symmetric.
- Asymmetric costs change the threshold, and sometimes the whole approach. Module 17 returns to this.

> ON THE EXAM
>
> A prediction nobody acts on differently is a prediction with no value, whatever its accuracy score.

Signal to approach

WHAT THE SCENARIO SAYS	WHERE TO LAND
No ML expertise, common task	AI service
Language generation or summarisation	Bedrock foundation model
Company documents must inform the answer	Bedrock with retrieval
Specific format or tone, examples available	Fine-tune, module 11
Tabular prediction, labelled rows	Built-in algorithm, usually XGBoost
Image task, some labelled images	JumpStart, fine-tuned
Regulator needs per-decision explanations	Interpretable model plus SHAP, not an FM
Novel problem, specialists, lots of labels	Custom training

ON THE EXAM

Two rows override the rest: an explainability requirement, and a hard latency budget. Both eliminate options outright.

When the answer is no model at all

- If the rule can be written down, **write the rule**. It is faster, cheaper and always explainable.
- A model that is right 94 percent of the time can be worse than a rule that is right 90 percent, if the rule is auditable.
- **Assessing feasibility is a named skill** in the exam guide. This is a legitimate answer.
- Also ask whether existing analytics already answer the question. A SQL query is not a failure.
- And whether anyone will act on the prediction. An unused prediction has no value at any accuracy.

> ON THE EXAM
>
> Module 01 made this point. It is repeated here because under exam pressure people forget that 'no model' is on the menu.

How cost behaves at each rung

APPROACH	COST SHAPE
AI service	Per request. No fixed cost. Scales linearly with usage
Bedrock on-demand	Per token, in and out. No idle cost, and no ceiling either
Bedrock provisioned	Per model unit-hour. Predictable, and paid whether used or not
Built-in algorithm	Training compute once, plus endpoint hours forever
JumpStart fine-tune	The same, plus the fine-tuning job
Custom training	The same, plus salaries. Usually the largest line and never on the bill

ON THE EXAM

Per-request pricing looks expensive at low volume and is not. Endpoint hours look cheap and run all night.

Total cost of ownership

- **The AWS bill is the small number.** Engineering time is usually the large one.
- A custom model needs retraining, monitoring, on-call ownership and someone who understands it.
- That person leaving is a real operational risk, and it has no line on any invoice.
- Managed options move that burden to AWS. That is what you are buying, more than compute.
- 'LEAST operational overhead' is asking about this cost, not the bill.

> ON THE EXAM
>
> When AWS says operational overhead, it means the work a human does on an ordinary day. That is the expensive part.

How hard is each choice to undo?

DECISION	COST OF CHANGING YOUR MIND
Which AI service to call	Low. Swap the API call
Which foundation model	Low. One parameter, then re-test prompts
Prompt design	Very low. That is the point of trying it first
Embedding model	High. Re-embed the entire corpus, module 09
Chunking strategy	High. New data source and re-ingest, module 10
Fine-tuning on a corpus	High. Retraining, and no un-training
Custom architecture	Highest. Months of work, and a team shaped around it

ON THE EXAM

Reversibility is a legitimate reason to prefer a lower rung: it keeps the expensive decisions in front of you, not behind.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Six months in, still no model in production	Started too high on the ladder
Model works, regulator rejects it	Interpretability requirement discovered late
Quality is fine, endpoint too slow	Foundation model against a millisecond budget
Bedrock cost rising without more users	Prompts growing, or too many retrieved chunks
Model ID stopped working	The model moved to Legacy or was removed
Custom model barely beats the AI service	The task was common. The ladder was skipped

ON THE EXAM

The first and last rows are the same mistake seen at different times: building what could have been bought.

Climb only as far as the requirement forces you, and **let the constraints do the eliminating**.

> A retailer receives around 20,000 product reviews a week and wants each classified as positive, negative or neutral, with the products and features mentioned extracted.
>
> The engineering team are backend developers with no machine learning experience. There is no labelled data. They want something in production within a fortnight.

What should they use?

A Train an XGBoost classifier on reviews after labelling a sample with Ground Truth

B Fine-tune a foundation model on Bedrock for sentiment and entity extraction

C Deploy a JumpStart text classification model and fine-tune it on the reviews

D Call Amazon Comprehend for sentiment and entity detection

◆ PAUSE AND DECIDE

What should they use?

A Train an XGBoost classifier on reviews after labelling a sample with Ground Truth

Weeks of labelling, ML expertise the team does not have, and Ground Truth is closed to new customers. Three separate blocks.

B Fine-tune a foundation model on Bedrock for sentiment and entity extraction

Fine-tuning needs labelled prompt-completion pairs, and there are none. Prompting an FM could work; fine-tuning cannot.

C Deploy a JumpStart text classification model and fine-tune it on the reviews

Fine-tuning again requires labels. It also hands the team an endpoint to own, which they have no capacity to run.

D Call Amazon Comprehend for sentiment and entity detection

Both requirements are named Comprehend features. No training, no labels, no endpoint, and a backend developer can ship it this week.

THE SIGNAL

Three constraints, each fatal on its own: no ML experience, no labelled data, two weeks. Only the bottom rung survives all three.

A bank must build a model that approves or declines small business loan applications from 40 structured financial fields. They hold 900,000 historical decisions with outcomes.

The regulator requires that for any individual decline, the bank can state which factors drove it and by how much. A written narrative explanation is not sufficient.

Which approach fits?

A. A foundation model on Bedrock, prompted to decide and explain its reasoning

B. An XGBoost model with per-prediction SHAP feature attributions

C. A deep neural network trained in script mode for maximum accuracy

D. Amazon Fraud Detector, configured for loan decisioning

◆ PAUSE AND DECIDE

Which approach fits?

A A foundation model on Bedrock, prompted to decide and explain its reasoning

It produces a fluent narrative, which the scenario explicitly rules out. A stated reason is not evidence of the actual computation.

B An XGBoost model with per-prediction SHAP feature attributions

Tabular data with 900,000 labelled rows is XGBoost's case exactly, and SHAP gives the per-decision attribution with magnitude the regulator wants.

C A deep neural network trained in script mode for maximum accuracy

Unlikely to beat gradient boosting on 40 tabular fields, and its attributions are approximations. It buys explainability risk for no accuracy gain.

D Amazon Fraud Detector, configured for loan decisioning

Built for transaction fraud, not credit decisioning, and it does not give the regulated per-decision attribution required here.

THE SIGNAL

'A written narrative is not sufficient' is the sentence doing the work. It rules out the fluent option specifically.

> A law firm wants 40-page contracts summarised into a one-page brief following the firm's own house structure. Partners have described the structure but nobody has written example briefs.
>
> There is no deadline pressure, and the firm is willing to iterate. They ask what to build first.

What is the right first step?

A. Prompt a Bedrock foundation model, specifying the house structure in the prompt

B. Fine-tune a Bedrock model on the firm's contracts and briefs

C. Run continued pre-training on the firm's contract archive

D. Build a knowledge base over the contracts and query it

◆ PAUSE AND DECIDE

What is the right first step?

A Prompt a Bedrock foundation model, specifying the house structure in the prompt

The structure has been described, so it can be written into a prompt. Prompt engineering is the cheapest customisation and costs nothing to try.

B Fine-tune a Bedrock model on the firm's contracts and briefs

There are no briefs. Fine-tuning needs prompt-completion pairs, and the completions do not exist yet.

C Run continued pre-training on the firm's contract archive

Teaches vocabulary the model largely has. The problem is output structure, not comprehension of legal English.

D Build a knowledge base over the contracts and query it

Retrieval finds relevant passages across a corpus. Here the whole contract is already the input, so there is nothing to retrieve.

THE SIGNAL

'Nobody has written example briefs' removes fine-tuning. 'The whole document is the input' removes retrieval. Prompting is what is left.

A manufacturer wants to detect six named surface defects on metal panels from production-line photographs. They have 8,000 labelled images, roughly balanced across the six classes.

An engineer proposes designing and training a convolutional network from scratch in script mode, arguing that their defects are unlike anything in a public dataset.

How should you respond?

A Agree. Novel defect types justify a purpose-built architecture

B Redirect to Amazon Rekognition, which detects objects in images

C Start with a pre-trained vision model from JumpStart, fine-tuned on the 8,000 images

D Prompt a multimodal foundation model on Bedrock with the defect descriptions

◆ PAUSE AND DECIDE

How should you respond?

A. Agree. Novel defect types justify a purpose-built architecture

The defects are novel; edges, textures and gradients are not. Training from scratch on 8,000 images discards pre-training that transfers directly.

B. Redirect to Amazon Rekognition, which detects objects in images

Rekognition detects general objects and scenes. Six specific surface defects are not in its label set, so it cannot do this task.

C. Start with a pre-trained vision model from JumpStart, fine-tuned on the 8,000 images

Transfer learning: the base model already knows visual primitives, so 8,000 images is enough to learn six specific classes on top.

D. Prompt a multimodal foundation model on Bedrock with the defect descriptions

Plausible for a prototype, and weak for a production line: no measurable accuracy target, higher latency, and per-image cost at manufacturing volume.

THE SIGNAL

8,000 images is a lot for fine-tuning and very few for training from scratch. The number itself points at the rung.

What to take from this module

- **Climb from the bottom.** Stop at the first rung that meets the requirement.
- **What data you hold** eliminates more options than anything else. Read it first.
- **Explainability and latency override preference.** Both can rule out foundation models outright.
- **Tabular with labels is still XGBoost.** Foundation models did not change that.
- **Learn selection criteria, not model names.** The Bedrock catalogue churns constantly.

> ON THE EXAM
>
> Module 13 goes down one rung and covers the built-in algorithms themselves, organised by problem shape.